

记忆系统的实现思考

记忆分类及设计方案

主流的记忆分类:

- working_memory: 短期近N轮
- episodic_memory: 情节的完整性,用户在什么时候说过什么,任务成功/失败原因
- semantic_memory: 事实、知识、概念、关系、偏好
- procedure_memory: 适用于agent, 完成某些类似任务的规范流程, 经过多次确认, 避免单次倾斜

记忆片段核心指标:

- 时间: 考虑衰减
- 重要性: 影响召回
- 置信度: 可信性

使用4类记忆推演记忆系统设计

Working Memory: 当前任务/当前会话的短期上下文
Episodic Memory: 过去发生过的事件、经历、轨迹
Semantic Memory: 事实、知识、概念、关系、偏好
Procedural Memory: 规则、技能、流程、策略、提示词、工具使用经验

- (1)利用内存级history记录Working Memory, 比如设定len=20
- (2)使用llm prompt1 对于batch提取, 比如每batch=10提取一次 Episodic Memory, 存向量库, 存mysql表。将主题+内容统一语义化(vector), 具体内容。注意向量库仅存主题, 进行相似召回
- (3)使用llm prompt2对于batch提取, 比如每batch=10提取一次Semantic Memory 存向量库, 存mysql表, 存graph。主要涉及具体的偏好, 事实, 实体关系等
- (4)使用llm prompt3 对于batch提取, 比如每batch=10提取一次 Procedural Memory, 注意倾斜 存向量库, 存mysql表, 主要涉及主题(vector), 具体内容。注意向量库仅存主题, 进行相似召回

词频检索方案

TF-IDF及BM25

- 1.TF-IDF 是一种传统文本检索方法, 存在一张词表, 通过统计一些高质量词频, 主要关注频率及稀有度。
 - (1)TF-IDF 会把每段文本转换成一个稀疏向量。
 - (2)把当前 query 也转成 TF-IDF 向量
 - (3)计算 query 和历史片段的余弦相似度
- 2.BM25
 - 对于TF-IDF的增强, 增加词饱和度和文档长度惩罚机制,
 - 同时对query,不通过计算文档余弦相似度,
 - 而是通过对 query 中每个词, 在文档中计算贡献分数最后求和

测试集的构建过程

- 1.业务场景
- 2.记忆类别分析
- 3.提取方式对比
 - 批次, 重叠度overlap
- 4.提取准确度评估
 - 人工提取记忆集合
 - 提取prompt调优及llm使用选型
 - 对比准确率、漏提率、多提率
- 5.召回准确度
 - 真实、构造query/改写prompt调优
 - 人工筛查query召回正确记忆集合
 - 向量相似度检索llm的记忆topN
 - 进行校准
- 6.最终效果
 - 数据集合:10k用户/500k对话,
 - 长留存:3留-7留, 提升5%

记忆系统的设计方案

记忆提取及落库

- 1.summary近期会话总结
 - (1)通过时间衰减策略+话题转换策略:
 - 时间策略衰减
 - 小时:1,2,6
 - 话题转换策略:
 - token jaccard (jaccard相似度: $A \cap B / A \cup B$)
 - ngram jaccard
 - 结构信号 jaccard: 比如 summary_text 与 query_text 都包含如:同一http,同一日期等。
 - 最终数据方案
 - <1小时:完整数据
 - 1-2 + 话题确认转换策略: 直接reset
 - 1-2 + 话题未转换: 完整数据
 - >2小时:部分切片事件[:N]
 - >6小时:reset
 - (2)使用llm对summary+message聚合
- 2.semantic记忆
 - 记忆的提取prompt1、相似记忆检索、记忆更新prompt2:UPDATE/ADD/DEL
 - milvus,mysql,graph(实体关系)
 - graph里: 实体与关系一定要定义边界
 - 通用实体类型
 - User
 - Organization

Person
Project
Product
Feature
Task
Concept
常见关系类型
WORKS_AT
OWNS
ASKED_ABOUT
PREFERS
DISLIKES
REQUIRES
DEPENDS_ON
HAS_COMPONENT

3.日程记忆

对逐个{role=user}消息通过embedding模型生成vector，
将vector与构造日程positive/negative cases，通过embedding进行语义匹配，这种方式可解决多语言，有一定的泛化能力

达到阈值才会进一步对message进行llm提取

```
positive_cases = [  
    "明天下午三点提醒我开会",  
    "帮我记住下周一上午有例会",  
    "remind me about tomorrow's meeting",  
    "I have an appointment next Monday morning",  
    "help me arrange a meeting next week",  
]  
negative_cases = [  
    "我今天心情不太好",  
    "我有点累想睡觉",  
    "哈哈你真会说话",  
    "that is funny",  
    "thanks for listening",  
]
```

将提取的结果，写入到mysql,可按照crontab提前10分钟，二次确认后写入mq触发，注意routine取消场景。

4.user/agentL2画像槽位

根据importance重要性等级，优先聚合high+的记忆片段

记忆检索

1.前处理:

根据规则集合，过滤非必要的改写

根据positive/negative cases，通过embedding进行语义匹配，筛掉负向query

```
positive_cases = [  
    "你还记得我们之前聊过什么吗",
```

```

"请总结一下我们最近讨论的重点",
"我之前提过什么安排和计划",
"what did we talk about before",
"can you summarize what we discussed earlier",
"did I mention any plans or preferences before",
]
negative_cases = [
    "好的收到谢谢",
    "哈哈哈哈",
    "我现在很难过很崩溃",
    "ok thanks got it",
    "haha lol",
    "i feel sad and tired",
]

```

2.进行query改写及查询:

通过summary及其policy:reset/decay,构造prompt, 从以下维度改写:

- semantic_query: 语义增强改写, 保留当前 query 的核心意图
- temporal_query: 时间线增强改写, 只有存在明确时间线线索时才填写
- entity_query: 实体增强改写, 只有存在明确人物、对象、事件、承诺才填写

并发查询并,对每个query进行limit=5

最终通过score/importance/时间奖励[越新的消息, 有一定的奖励额度time_bonus]
进行排序取topK=10

3.二次重排序, 最终取top5:

对每个query、每个memory进行遍历, 计算memory的分数

```

for query_list = [raw_query, semantic_query, entity_query, temporal_query]
    for retrieve_memroies

```

分数计算规则:

base_score[召回评分、重要性、时间奖励]: 0.62

lexical_score (词汇相似分) : 0.22

token + jaccard

ngram + jaccard

time_decay_score(时间衰减): 0.13

containment_bonus(query,与memory对应召回的query, 匹配): 0.03

RRF扩展: 学习RRF重排方案(reciprocal rank fusion)

对于比如 vector、graph不同维度的召回进行统一排序时,

使用rank倒数重排, 代码如下:

```

def rrf_score(ranks, k=60):
    score = 0
    for rank in ranks:
        score += 1 / (k + rank)
    return score

```

1.能够避免score尺度不一致, 需额外增加归一化、

2.用倒数就是为了表达递减效应, 避免正数排序的线性,

比如:第1名和第2名的差异, 通常比第99名和第100名的差异更有价值